

PLEX 多智能体系统接入开发提示词

请为 PLEX 接入多智能体协同推理能力。本次接入必须围绕 PLEX 最新定位展开：

PLEX 是面向 Python 初学者的自适应学习系统。

不要把多智能体做成泛聊天系统，也不要做成复杂炫技平台。多智能体必须服务于 Python 初学者的学习闭环。

一、技术选型

优先使用：

CrewAI

GitHub：

<https://github.com/crewAIInc/crewAI>

如果当前项目后端不是 Python，也可以先用 mock API 模拟多智能体结果，但接口结构必须为后续 CrewAI 接入预留。

暂时不要优先接：

AutoGen
LangGraph
VoltAgent

原因：

1. 当前阶段需要稳定可演示；
2. CrewAI 更适合快速定义角色型智能体；
3. PLEX 现在面向 Python 初学者，CrewAI 的 Python 生态更容易讲清楚；
4. LangGraph 可以作为第二阶段升级，用于复杂分支和有状态学习流。

二、总体目标

请实现一个面向 Python 初学者的多智能体学习支持流程：

学生提交 Python 代码 / 完成练习
↓
学习诊断智能体分析当前问题
↓
代码分析智能体分析代码错误
↓
知识图谱智能体定位相关知识点
↓
路径推荐智能体推荐下一步学习内容
↓
反馈生成智能体生成学生可理解的反馈
↓
教师助理智能体为教师端生成教学建议

系统前端不需要展示 Agent 互相聊天的过程，而是展示：

1. 当前诊断结果
2. 相关知识点
3. 推荐学习路径
4. 分层提示
5. 教师端教学建议
6. 管理员端智能体运行状态

三、推荐文件结构

如果项目支持 Python 后端，请新增：

```
backend/  
  agents/  
    __init__.py  
    crew.py  
    learning_diagnosis_agent.py  
    code_analysis_agent.py  
    knowledge_graph_agent.py  
    path_recommendation_agent.py  
    feedback_agent.py  
    teacher_assistant_agent.py  
  
  services/  
    agent_service.py  
  
  routes/
```

```
agent_routes.py

schemas/
agent_schema.py
```

前端新增或调整：

```
src/
  api/
    agentApi.ts

  components/
    agent/
      PlexAgentStatusCard.vue
      PlexAgentFlowPanel.vue
      PlexDiagnosisPanel.vue
      PlexLearningAdvicePanel.vue
      PlexTeacherSuggestionPanel.vue

  views/
    student/
      StudentAIAssistant.vue
      StudentPracticeFeedback.vue

    teacher/
      TeacherAgentInsight.vue

    admin/
      AdminAgentOrchestration.vue
```

如果当前项目是纯前端或后端暂未完善，请先实现 mock service：

```
src/mock/
agentMock.ts
```

四、智能体角色设计

请定义 6 个核心智能体。

1. 学习诊断智能体 LearningDiagnosisAgent

职责：

判断学生当前学习问题，识别薄弱知识点和错误类型。

输入：

```
{
  studentId: string
  exerciseId: string
  knowledgePoints: string[]
  answerStatus: 'correct' | 'wrong' | 'partial'
  attemptCount: number
  errorMessage?: string
  recentMistakes?: string[]
}
```

输出：

```
{
  weakPoints: string[]
  errorType: 'syntax' | 'logic' | 'concept' | 'input_output' | 'unknown'
  diagnosis: string
  confidence: number
}
```

示例输出：

学生可能已经理解循环结构，但对 range 的边界理解不稳定，导致少统计一次。

2. 代码分析智能体 CodeAnalysisAgent

职责：

分析学生 Python 代码中的错误，并给出初学者能理解的解释。

输入：

```
{
  code: string
  stdin?: string
  stdout?: string
  stderr?: string
  expectedOutput?: string
}
```

输出：

```
{
  codeIssueSummary: string
  possibleCause: string
  fixDirection: string
  relatedConcepts: string[]
}
```

要求：

1. 不要直接给完整答案；
2. 优先解释错误原因；
3. 给出修改方向；
4. 适合 Python 初学者；
5. 语言要温和，不要打击学生。

示例输出：

你的循环条件可能没有覆盖到最后一个数字。可以检查 `range` 的结束值是否比你预期少 1。

3. 知识图谱智能体 KnowledgeGraphAgent

职责：

根据学生错误和题目知识点，定位相关知识节点和前置知识。

输入：

```
{
  currentKnowledgePoints: string[]
  weakPoints: string[]
}
```

```
    errorType: string
}
```

输出：

```
{
  relatedNodes: string[]
  prerequisiteNodes: string[]
  recommendedReviewNodes: string[]
  graphReason: string
}
```

知识范围必须限定在 Python 初学者知识体系：

```
print 输出
变量
输入 input
数据类型
类型转换
算术运算
比较运算
逻辑运算
if 条件分支
for 循环
while 循环
range
break / continue
列表
字典
字符串
函数
文件基础
异常基础
最大最小值
计数统计
线性查找
简单排序思想
```

不要推荐过深内容，例如：

```
动态规划
图论
高级数据结构
装饰器
```

元类
异步编程
复杂面向对象

4. 路径推荐智能体 PathRecommendationAgent

职责：

根据诊断结果和知识图谱结果，推荐下一步学习路径。

输入：

```
{
  weakPoints: string[]
  prerequisiteNodes: string[]
  currentStage: number
  completedExercises: string[]
}
```

输出：

```
{
  nextKnowledgePoint: string
  recommendedExercises: string[]
  reviewPlan: string[]
  estimatedDifficulty: 'easy' | 'medium'
}
```

推荐原则：

1. 优先补前置知识；
2. 不要跳到过难内容；
3. 每次最多推荐 1-3 个练习；
4. 推荐理由要简洁；
5. 面向 Python 初学者。

5. 反馈生成智能体 FeedbackAgent

职责：

将前面智能体的分析结果转化为学生端可读的学习反馈。

输入：

```
{
  diagnosis: string
  codeIssueSummary?: string
  fixDirection?: string
  nextKnowledgePoint: string
  recommendedExercises: string[]
}
```

输出：

```
{
  shortFeedback: string
  stepHints: string[]
  encouragement: string
  nextAction: string
}
```

要求：

1. 不要直接给完整代码答案；
2. 分层提示，最多 3 步；
3. 语言适合初学者；
4. 反馈要具体，不要空泛；
5. 最后给出下一步行动。

示例：

你已经掌握了循环的大致写法，现在主要问题是边界值判断。先观察 range 的起点和终点，再用一个小例子手动推一遍循环次数。

6. 教师助理智能体 TeacherAssistantAgent

职责：

根据班级共性错误和学生薄弱点，为教师生成教学建议。

输入：

```
{
  classId: string
  weakPointStats: {
    knowledgePoint: string
    count: number
  }[]
  commonErrorTypes: string[]
  recentExercises: string[]
}
```

输出：

```
{
  classSummary: string
  teachingSuggestions: string[]
  interventionGroups: {
    groupName: string
    students: string[]
    focus: string
  }[]
}
```

示例输出：

本班近期在 for 循环与 range 边界理解上出现较多错误，建议下节课使用可视化方式演示 range(1, n) 与 range(1, n+1) 的区别。

五、后端 API 设计

请新增以下接口。

1. 学生端单题诊断接口

POST /api/agents/student-diagnose

请求：

```

{
  studentId: string
  exerciseId: string
  code: string
  stdin?: string
  stdout?: string
  stderr?: string
  expectedOutput?: string
  knowledgePoints: string[]
  attemptCount: number
}

```

返回：

```

{
  diagnosis: {
    weakPoints: string[]
    errorType: string
    diagnosis: string
    confidence: number
  }
  codeAnalysis: {
    codeIssueSummary: string
    possibleCause: string
    fixDirection: string
    relatedConcepts: string[]
  }
  graphInsight: {
    relatedNodes: string[]
    prerequisiteNodes: string[]
    recommendedReviewNodes: string[]
    graphReason: string
  }
  recommendation: {
    nextKnowledgePoint: string
    recommendedExercises: string[]
    reviewPlan: string[]
    estimatedDifficulty: string
  }
  feedback: {
    shortFeedback: string
    stepHints: string[]
    encouragement: string
    nextAction: string
  }
}

```

```
}  
}
```

2. 教师端班级建议接口

POST /api/agents/teacher-suggestion

请求：

```
{  
  classId: string  
  weakPointStats: {  
    knowledgePoint: string  
    count: number  
  }[]  
  commonErrorTypes: string[]  
  recentExercises: string[]  
}
```

返回：

```
{  
  classSummary: string  
  teachingSuggestions: string[]  
  interventionGroups: {  
    groupName: string  
    students: string[]  
    focus: string  
  }[]  
}
```

3. 管理员端智能体状态接口

GET /api/agents/status

返回：

```
{
  agents: {
    id: string
    name: string
    role: string
    status: 'idle' | 'running' | 'success' | 'error'
    lastRunAt?: string
    avgLatency?: number
  }[]
}
```

六、前端展示要求

1. 学生端展示

在学生端 Python 练习结果页增加：

AI 学习反馈面板

包含：

1. 当前问题诊断
2. 代码错误解释
3. 相关知识点
4. 下一步推荐
5. 分层提示

不要展示复杂 Agent 内部推理过程。

组件建议：

PlexDiagnosisPanel.vue
PlexLearningAdvicePanel.vue

2. 教师端展示

在教师端班级分析页增加：

AI 教师建议

包含：

1. 班级共性问题总结
2. 高频薄弱知识点
3. 教学干预建议
4. 学生分组建议

组件建议：

PlexTeacherSuggestionPanel.vue

3. 管理员端展示

在管理员端智能体编排页面增加：

多智能体运行状态

包含：

1. 学习诊断智能体
2. 代码分析智能体
3. 知识图谱智能体
4. 路径推荐智能体
5. 反馈生成智能体
6. 教师助理智能体

每个 Agent 显示：

名称
职责
运行状态
最近运行时间
平均响应时间

可以配合现有 Vue Flow 画布展示流程：

学习诊断 → 代码分析 → 知识图谱定位 → 路径推荐 → 反馈生成

七、CrewAI 实现要求

如果后端可以使用 Python，请实现 CrewAI 基础结构。

1. 安装

```
pip install crewai
```

2. Agent 定义方式

每个 Agent 需要有：

```
role  
goal  
backstory  
task  
expected_output
```

3. 工作流方式

第一阶段使用顺序型流程：

```
LearningDiagnosisAgent  
↓  
CodeAnalysisAgent  
↓  
KnowledgeGraphAgent  
↓  
PathRecommendationAgent  
↓  
FeedbackAgent
```

教师端使用单独流程：

```
TeacherAssistantAgent
```

4. 输出格式

必须返回结构化JSON，不要返回难以解析的大段文本。

5. 失败处理

如果 CrewAI 调用失败，后端必须返回 mock fallback，不允许前端崩溃。

八、Mock fallback 要求

如果暂时无法真实接入 CrewAI，请先实现 mock 智能体服务。

mock 结果要真实可信，不能全是“暂无数据”。

示例：

```
{
  diagnosis: {
    weakPoints: ['range 边界', 'for 循环'],
    errorType: 'logic',
    diagnosis: '学生可能理解了循环结构，但对 range 的结束边界不稳定。',
    confidence: 0.86
  },
  codeAnalysis: {
    codeIssueSummary: '循环少执行了一次。',
    possibleCause: 'range 的结束值不包含在循环范围内。',
    fixDirection: '检查 range 的第二个参数是否需要加 1。',
    relatedConcepts: ['for 循环', 'range']
  },
  graphInsight: {
    relatedNodes: ['for 循环', 'range', '循环边界'],
    prerequisiteNodes: ['变量', '数字类型', '比较运算'],
    recommendedReviewNodes: ['range', 'for 循环'],
    graphReason: '该错误与循环次数和边界理解直接相关。'
  },
  recommendation: {
    nextKnowledgePoint: 'range 的边界规则',
    recommendedExercises: ['1 到 n 求和', '统计偶数个数'],
    reviewPlan: ['复习 range(start, end)', '手动列出循环变量变化', '完成 2 道循环边界练习'],
    estimatedDifficulty: 'easy'
  },
  feedback: {
    shortFeedback: '你已经会写 for 循环了，现在需要重点检查循环的结束位置。',
  }
}
```

```
    stepHints: [  
        '先写出 range 生成的每一个数字。',  
        '观察最后一个数字是否包含在循环中。',  
        '再对照题目要求检查是否少算了一次。'  
    ],  
    encouragement: '这个错误很常见，说明你已经进入真正理解循环的阶段了。',  
    nextAction: '建议先完成“1 到 n 求和”练习来巩固 range 的边界。'  
}  
}
```

九、和 Python 知识图谱的关系

多智能体必须围绕当前 Python 初学者知识图谱运行。

知识图谱节点包括但不限于：

```
print 输出  
变量  
数字类型  
字符串  
布尔值  
输入 input  
类型转换  
算术运算  
比较运算  
逻辑运算  
if 条件分支  
for 循环  
while 循环  
range  
break  
continue  
列表  
字典  
集合  
字符串方法  
函数  
参数  
返回值  
文件读取  
异常处理  
计数统计  
累加求和  
最大值最小值
```


线性查找
简单排序思想

不得推荐过深内容：

动态规划
图论
高级面向对象
装饰器
生成器
协程
元类
复杂算法竞赛题

十、UI 风格要求

请保持 PLEX 原有视觉风格。

学生端：

绿色主题
反馈语言温和
强调学习成长感

教师端：

橙色主题
强调教学支持和班级分析

管理员端：

蓝紫色科技主题
强调系统编排和运行观测

不要直接套用第三方 Agent 平台 UI。

十一、性能要求

多智能体功能不要导致页面卡顿。

要求：

1. Agent 请求必须异步；
2. 前端显示 loading skeleton；
3. 超过 15 秒给出超时提示；
4. 失败时返回 fallback 建议；
5. 不要在页面切换时继续更新已卸载组件；
6. 管理员端状态轮询间隔不要太短，建议 30 秒以上；
7. 不要在学生每次输入代码时触发 Agent，只在点击“获取 AI 反馈”或提交后触发。

十二、安全与教学边界

反馈生成时必须遵守：

1. 不直接给完整答案代码；
2. 优先给提示、思路、错误定位；
3. 保护初学者信心；
4. 不生成与题目无关内容；
5. 不推荐超纲知识；
6. 不替代教师最终判断；
7. 教师端建议要表述为“建议”，不要说成绝对结论。

十三、验收标准

完成后需要满足：

1. 学生提交 Python 代码后，可以获取 AI 学习反馈
 2. 反馈包括诊断、代码分析、知识点、路径推荐和分层提示
 3. 教师端可以生成班级教学建议
 4. 管理员端可以看到 6 个智能体及其状态
 5. 多智能体围绕 Python 初学者知识体系运行
 6. 不出现 C++ 或复杂算法竞赛内容
 7. CrewAI 不可用时 mock fallback 正常
 8. 页面不卡顿
 9. 不破坏 PLEX 原有 UI
 10. 输出结构化，便于前端渲染

十四、答辩展示描述

请在项目说明中保留以下表述：

PLEX 引入多智能体协同推理机制，但并非简单增加一个 AI 聊天入口，而是将 Python 初学者的学习支持过程拆解为多个专业智能体：学习诊断智能体负责识别薄弱点，代码分析智能体负责解释 Python 错误，知识图谱智能体负责定位前置知识，路径推荐智能体负责生成下一步学习建议，反馈生成智能体负责输出分层提示，教师助理智能体负责形成班级教学建议。多个智能体围绕“诊断—分析—定位—推荐—反馈—干预”的学习闭环协同工作，从而提升系统的个性化、自适应与可解释性。